Contents

Procurement and Warehouse Database Project Report

Abstract

This project documents a relational database system for a manufacturing company that manages procurement, supplier communication, warehouse receiving, inventory balances, approval routing, departmental budget control, quality inspections, and vendor claims. The implementation uses Microsoft SQL Server for the database layer and Streamlit for the user interface. The final solution combines normalized relational design, procedural SQL objects, role-based authorization, and live execution support for the full business flow from request creation to supplier claim handling.

1. User Requirements

The target organization is a medium-scale manufacturing company that buys raw materials, packaging materials, and maintenance items from multiple suppliers and receives them into more than one warehouse. The system must support three main user groups: procurement staff, warehouse staff, and reporting users. It must also support supervisory tasks such as approval control, budget monitoring, inspection logging, and supplier dispute tracking.

The primary user requirements are:

- store supplier master data, supplier contacts, and commercial attributes

- store employee, application user, and user-group information
- maintain material master data, material types, units of measure, specifications, and reorder levels
- track warehouse definitions and current material balances by warehouse
- allow employees to create purchase requests with material-level line items
- allow suppliers to submit offers for request items
- allow accepted offers to be converted into purchase orders
- record goods receipts and accepted versus rejected receipt quantities
- create and post material transactions that update stock balances
- route purchase requests and purchase orders through formal approval steps
- maintain monthly department budgets and consume budget through purchase commitments
- record quality inspections after receiving deliveries
- convert rejected or quarantined inspection quantities into vendor claims
- expose operational and analytical data through reusable views and functions
- enforce consistency with keys, checks, triggers, stored procedures, and transaction isolation levels
- provide a multi-module user interface that executes the main workflows directly against the database

From a retrieval perspective, procurement users need open-request lists, offer comparisons, pending approvals, budget status, and claim status. Warehouse users need inventory balances, reorder alerts, receipt history, inspection summaries, and receipt-level quality outcomes. Reporting users need read-only visibility into the full flow across procurement, control, and audit data.

2. Entity-Relationship Diagram

The database is centered on a procurement and warehouse domain rather than an academic records model. The structure is intentionally document-oriented and multi-stage. It includes master data, transactional documents, operational control layers, and post-receipt quality and claim processes.

For readability, the report shows the E-R design in layers:

- one overview figure for the full system structure
- one security and identity subsystem figure
- one procurement, approval, and budget subsystem figure
- one warehouse, inspection, and vendor-claim subsystem figure

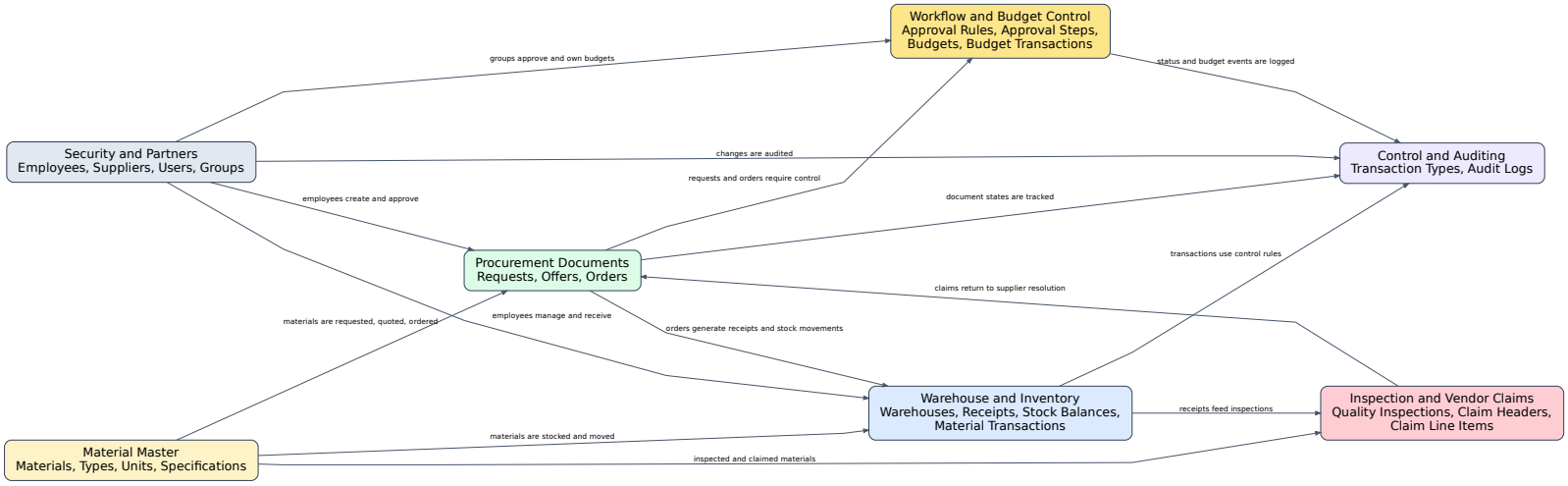


Figure 1: Overall E-R overview of the procurement, control, and warehouse database

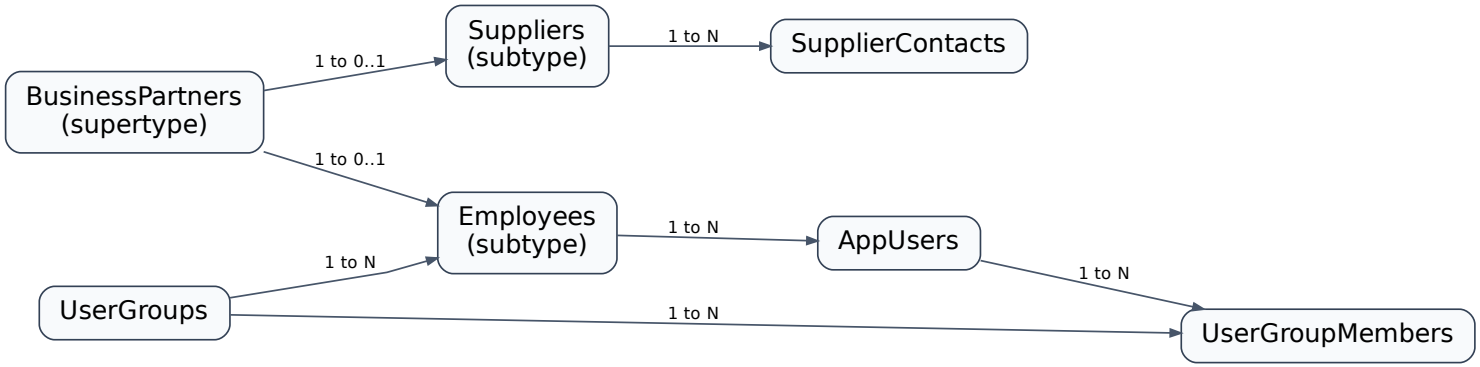


Figure 2: Security, personnel, inheritance, and supplier-contact subsystem

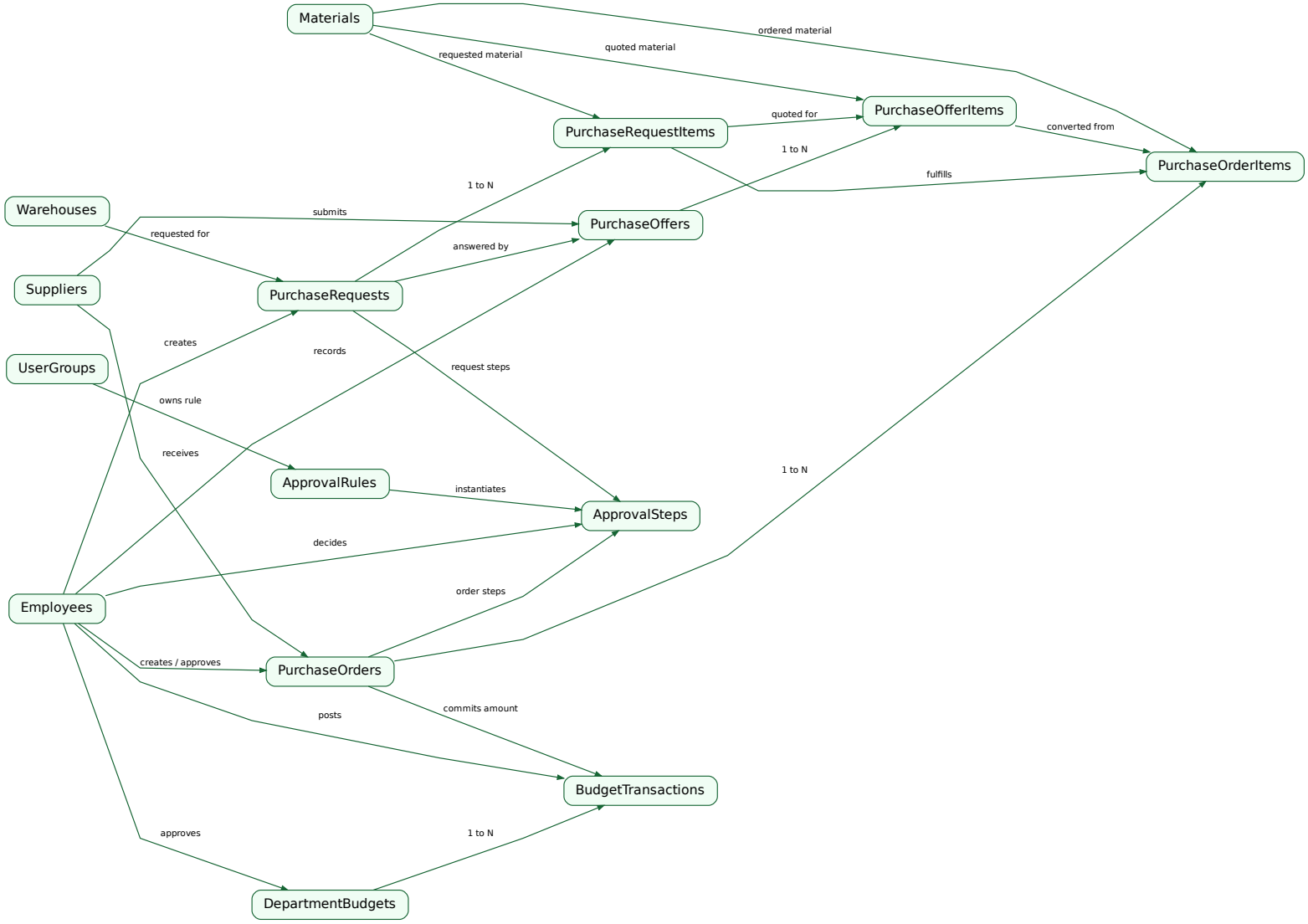


Figure 3: Procurement, approval workflow, and budget control subsystem

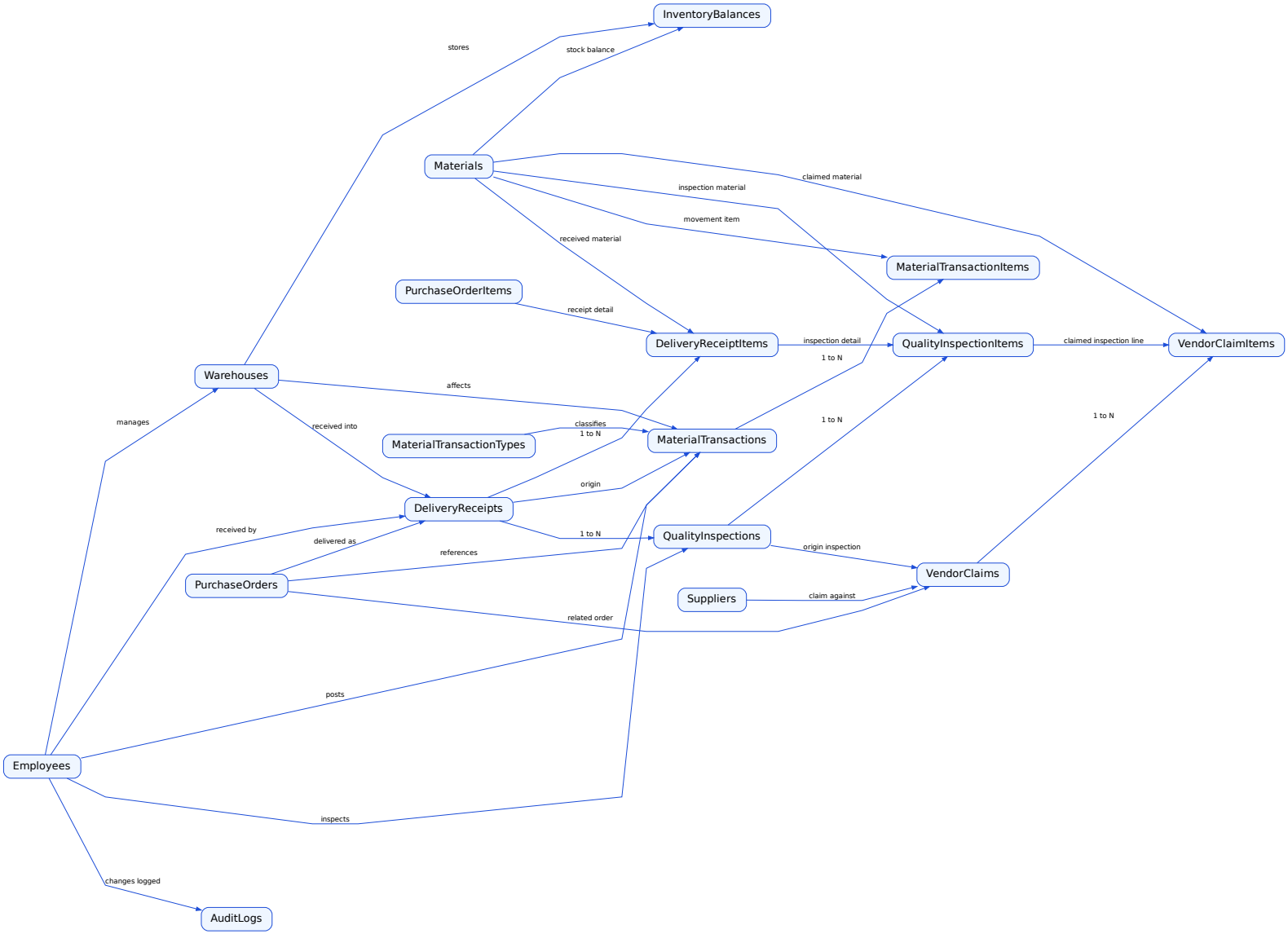


Figure 4: Warehouse, receiving, inspection, and vendor-claim subsystem

The main relationship groups are:

- `BusinessPartners` is the supertype for `Employees` and `Suppliers`
- `Materials`, `MaterialTypes`, `MeasurementUnits`, and `MaterialSpecifications` define the item master structure
- `PurchaseRequests`, `PurchaseOffers`, and `PurchaseOrders` form the commercial document chain
- `ApprovalRules` and `ApprovalSteps` form the approval workflow layer
- `DepartmentBudgets` and `BudgetTransactions` form the budget control layer
- `DeliveryReceipts`, `MaterialTransactions`, and `InventoryBalances` form the receiving and stock movement layer
- `QualityInspections` and `QualityInspectionItems` form the post-receipt inspection layer
- `VendorClaims` and `VendorClaimItems` form the supplier issue-resolution layer
- `AuditLogs` stores important trace evidence

3. Functional Dependencies

The main functional dependencies identified in the model are listed below. These dependencies guided the decomposition of the schema into normalized relations.

- `PartnerID -> PartnerType, LegalName, Email, Phone, IsActive, CreatedAt`
- `EmployeeID(PartnerID) -> EmployeeNo, FirstName, LastName, GroupID, HireDate, JobTitle, DepartmentName`
- `SupplierID(PartnerID) -> SupplierCode, TaxNumber, PaymentTermDays, PreferredSupplier, LastOfferDate`
- `UserID -> EmployeeID, UserName, UserEmail, PasswordHash, UserStatus, CreatedAt`
- `GroupID -> GroupName, GroupDescription, IsActive`
- `WarehouseID -> WarehouseCode, WarehouseName, WarehouseAddress, ManagerID, IsActive`
- `UnitID -> UnitCode, UnitName`
- `MaterialTypeID -> TypeCode, TypeName`
- `MaterialID -> MaterialCode, MaterialName, MaterialTypeID, BaseUnitID, ReorderLevel, StandardCost, LeadTimeDays, IsCritical, IsActive`
- `RequestID -> RequestNo, RequestedByEmployeeID, WarehouseID, RequestDate, NeededByDate, RequestStatus, PriorityCode, RequestNotes`
- `(RequestID, MaterialID) -> RequestedQuantity, ApprovedQuantity, IssuedQuantity, ItemStatus`
- `OfferID -> OfferNo, RequestID, SupplierID, CreatedByEmployeeID, OfferDate, ValidUntil, OfferStatus, EvaluationNotes`
- `(OfferID, RequestItemID) -> MaterialID, OfferedQuantity, UnitPrice, PromisedDeliveryDate, ItemStatus`
- `OrderID -> OrderNo, SupplierID, CreatedByEmployeeID, ApprovedByEmployeeID, OrderDate, RequiredDeliveryDate, OrderStatus, PaymentStatus, OrderNotes`

- (OrderID, RequestItemID) -> OfferItemID, MaterialID, OrderedQuantity, ReceivedQuantity, UnitPrice, ItemStatus
- ReceiptID -> ReceiptNo, OrderID, ReceivedWarehouseID, ReceivedByEmployeeID, InvoiceNo, CarrierName, ReceiptDate, ReceiptStatus, ReceiptNotes
- (WarehouseID, MaterialID) -> OnHandQuantity, ReservedQuantity, LastUpdated
- ApprovalRuleID -> TargetTable, StepNo, GroupID, RuleName, IsActive
- ApprovalID -> ApprovalRuleID, TargetTable, TargetRecordID, StepNo, AssignedGroupID, ApprovalStatus, DecisionByEmployeeID, DecisionDate
- BudgetID -> DepartmentName, BudgetYear, BudgetMonth, BudgetAmount, AlertThresholdPct, ApprovedByEmployeeID
- BudgetTransactionID -> BudgetID, RelatedOrderID, TransactionType, Amount, TransactionNotes, CreatedByEmployeeID, TransactionDate
- InspectionID -> InspectionNo, ReceiptID, InspectedByEmployeeID, InspectionDate, InspectionStatus, InspectionNotes
- (InspectionID, ReceiptItemID) -> MaterialID, AcceptedQuantity, RejectedQuantity, QuarantineQuantity, DefectCode, ResolutionStatus
- ClaimID -> ClaimNo, SupplierID, OrderID, InspectionID, ClaimDate, ClaimStatus, ClaimReason, CreatedByEmployeeID, SettlementAmount
- (ClaimID, InspectionItemID) -> MaterialID, ClaimedQuantity, UnitPrice, ResolutionCode

These dependencies justify the separation between master entities, document headers, document lines, workflow tables, budget ledgers, inspection records, claim records, and warehouse balances.

4. Normalization

The final schema was normalized to Third Normal Form (3NF) and evaluated against Boyce-Codd Normal Form (BCNF). The design objective was to remove redundancy, avoid update anomalies, and keep each fact in the table where it belongs.

Example A: Relation not in 3NF

Assume an early design stored order and supplier data together in the following relation:

PurchaseOrderLine_Bad(OrderID, OrderDate, SupplierID, SupplierName, SupplierTaxNumber, MaterialID, MaterialName, OrderedQty, UnitPrice)

The dependencies are:

- OrderID -> OrderDate, SupplierID
- SupplierID -> SupplierName, SupplierTaxNumber
- MaterialID -> MaterialName
- (OrderID, MaterialID) -> OrderedQty, UnitPrice

This relation is not in 3NF because SupplierName, SupplierTaxNumber, and MaterialName are transitively dependent on non-key determinants. The resulting problems are:

- supplier details must be updated in multiple rows
- deleting the last order line may accidentally remove supplier knowledge
- inserting a new supplier without an order line is impossible

The decomposition is:

- `PurchaseOrders(OrderID, OrderDate, SupplierID, ...)`
- `Suppliers(SupplierID, SupplierName, SupplierTaxNumber, ...)`
- `Materials(MaterialID, MaterialName, ...)`
- `PurchaseOrderItems(OrderID, MaterialID, OrderedQty, UnitPrice, ...)`

This decomposition removes the transitive dependency and satisfies 3NF.

Example B: Relation in 3NF but not in BCNF

Consider the relation:

`WarehouseTypeSpecialist(WarehouseID, MaterialTypeID, SpecialistEmployeeID)`

Suppose the business rules are:

- each `(WarehouseID, MaterialTypeID)` pair has one specialist
- each specialist is assigned to one material type

The dependencies are:

- `(WarehouseID, MaterialTypeID) -> SpecialistEmployeeID`
- `SpecialistEmployeeID -> MaterialTypeID`

This relation may satisfy 3NF if `MaterialTypeID` is treated as prime in candidate-key analysis, but it violates BCNF because `SpecialistEmployeeID` is not a superkey. The BCNF decomposition is:

- `WarehouseSpecialists(WarehouseID, SpecialistEmployeeID)`
- `SpecialistTypes(SpecialistEmployeeID, MaterialTypeID)`

Normalization Status of the Final Design

The implemented schema satisfies normalization goals because:

- header and line-item data are separated
- employee and supplier identity data are stored once in a supertype-subtype structure
- request, offer, order, receipt, inspection, and claim facts are separated by document layer
- many-to-many relationships are resolved through associative tables such as `UserGroupMembers`
- monthly department budgets are separated from budget transactions
- approval rules are separated from approval instances
- unique constraints prevent duplicate material lines or repeated claim-item references in the same document context

The final design therefore avoids the principal insertion, deletion, and update anomalies of the non-normalized examples.

5. Database Schema

The schema is organized into logical subsystems to keep operational responsibilities clear.

5.1 Security and Personnel

- UserGroups(GroupID PK, GroupName, GroupDescription, IsActive)
- BusinessPartners(PartnerID PK, PartnerType, LegalName, Email, Phone, IsActive, CreatedAt)
- Employees(PartnerID PK/FK, EmployeeNo, FirstName, LastName, GroupID FK, HireDate, JobTitle, DepartmentName)
- Suppliers(PartnerID PK/FK, SupplierCode, TaxNumber, PaymentTermDays, PreferredSupplier, LastOfferDate)
- AppUsers(UserID PK, EmployeeID FK, UserName, UserEmail, PasswordHash, UserStatus, CreatedAt)
- UserGroupMembers(UserID FK, GroupID FK, AssignedAt, PK(UserID, GroupID))
- SupplierContacts(ContactID PK, SupplierID FK, ContactName, ContactTitle, ContactEmail, ContactPhone, IsPrimary, IsActive)

5.2 Material Master

- MeasurementUnits(UnitID PK, UnitCode, UnitName)
- MaterialTypes(MaterialTypeID PK, TypeCode, TypeName)
- Materials(MaterialID PK, MaterialCode, MaterialName, MaterialTypeID FK, BaseUnitID FK, ReorderLevel, StandardCost, LeadTimeDays, IsCritical, IsActive)
- MaterialSpecifications(MaterialID PK/FK, DrawingNo, RevisionNo, PhotoReference, QualityNotes, MainSubstituteMaterialID FK)

5.3 Warehouse and Inventory

- Warehouses(WarehouseID PK, WarehouseCode, WarehouseName, WarehouseAddress, ManagerID FK, IsActive)
- InventoryBalances(InventoryID PK, WarehouseID FK, MaterialID FK, OnHandQuantity, ReservedQuantity, LastUpdated, UNIQUE(WarehouseID, MaterialID))
- MaterialTransactionTypes(TransactionTypeID PK, TypeCode, TypeName, StockEffect, RequiresDocument)
- MaterialTransactions(TransactionID PK, TransactionNo, TransactionTypeID FK, WarehouseID FK, RelatedRequestID FK, RelatedOrderID FK, ReferenceReceiptID FK, CreatedByEmployeeID FK, TransactionDate, PostingStatus)
- MaterialTransactionItems(TransactionItemID PK, TransactionID FK, MaterialID FK, Quantity, UnitCost)

5.4 Procurement Documents

- PurchaseRequests(RequestID PK, RequestNo, RequestedByEmployeeID FK, WarehouseID FK, RequestDate, NeededByDate, RequestStatus, PriorityCode, RequestNotes, CreatedAt)
- PurchaseRequestItems(RequestItemID PK, RequestID FK, MaterialID FK, RequestedQuantity, ApprovedQuantity, IssuedQuantity, ItemStatus, UNIQUE(RequestID, MaterialID))
- PurchaseOffers(OfferID PK, OfferNo, RequestID FK, SupplierID FK, CreatedByEmployeeID FK, OfferDate, ValidUntil, OfferStatus, EvaluationNotes, CreatedAt)
- PurchaseOfferItems(OfferItemID PK, OfferID FK, RequestItemID FK, MaterialID FK, OfferedQuantity, UnitPrice, PromisedDeliveryDate, ItemStatus, UNIQUE(OfferID, RequestItemID))
- PurchaseOrders(OrderID PK, OrderNo, SupplierID FK, CreatedByEmployeeID FK, ApprovedByEmployeeID FK, OrderDate, RequiredDeliveryDate, OrderStatus, PaymentStatus, OrderNotes, CreatedAt)
- PurchaseOrderItems(OrderItemID PK, OrderID FK, OfferItemID FK, RequestItemID FK, MaterialID FK, OrderedQuantity, ReceivedQuantity, UnitPrice, ItemStatus, UNIQUE(OrderID, RequestItemID))

5.5 Approval and Budget Control

- ApprovalRules(ApprovalRuleID PK, TargetTable, StepNo, GroupID FK, RuleName, IsActive)
- ApprovalSteps(ApprovalID PK, ApprovalRuleID FK, TargetTable, TargetRecordID, StepNo, AssignedGroupID FK, ApprovalStatus, DecisionByEmployeeID FK, DecisionDate, DecisionNotes, CreatedAt)
- DepartmentBudgets(BudgetID PK, DepartmentName, BudgetYear, BudgetMonth, BudgetAmount, AlertThresholdPct, ApprovedByEmployeeID FK, CreatedAt)
- BudgetTransactions(BudgetTransactionID PK, BudgetID FK, RelatedOrderID FK, TransactionType, Amount, TransactionNotes, CreatedByEmployeeID FK, TransactionDate)

5.6 Delivery, Inspection, and Claims

- DeliveryReceipts(ReceiptID PK, ReceiptNo, OrderID FK, ReceivedWarehouseID FK, ReceivedByEmployeeID FK, InvoiceNo, CarrierName, ReceiptDate, ReceiptStatus, ReceiptNotes)
- DeliveryReceiptItems(ReceiptItemID PK, ReceiptID FK, OrderItemID FK, MaterialID FK, ReceivedQuantity, AcceptedQuantity, RejectedQuantity, UNIQUE(ReceiptID, OrderItemID))
- QualityInspections(InspectionID PK, InspectionNo, ReceiptID FK, InspectedByEmployeeID FK, InspectionDate, InspectionStatus, InspectionNotes, CreatedAt)
- QualityInspectionItems(InspectionItemID PK, InspectionID FK, ReceiptItemID

FK, MaterialID FK, AcceptedQuantity, RejectedQuantity, QuarantineQuantity, DefectCode, ResolutionStatus, UNIQUE(InspectionID, ReceiptItemID))

- VendorClaims(ClaimID PK, ClaimNo, SupplierID FK, OrderID FK, InspectionID FK, ClaimDate, ClaimStatus, ClaimReason, CreatedByEmployeeID FK, SettlementAmount)
- VendorClaimItems(ClaimItemID PK, ClaimID FK, InspectionItemID FK, MaterialID FK, ClaimedQuantity, UnitPrice, ResolutionCode, UNIQUE(ClaimID, InspectionItemID))

5.7 Audit and Monitoring

- AuditLogs(AuditLogID PK, TableName, RecordID, ActionName, OldValue, NewValue, ChangedByEmployeeID FK, ChangedAt)

Overall, the current implementation contains 33 tables.

6. Database Implementation

The implementation is organized into three SQL scripts:

- sql/01_schema_and_security.sql
- sql/02_business_logic.sql
- sql/03_seed_and_demo_queries.sql

The implementation includes:

- database creation and reset logic
- all table definitions
- primary keys, foreign keys, check constraints, unique constraints, and filtered unique indexes
- security roles and demo users
- scalar functions and table-valued functions
- analytical and operational views
- procedural transactions and supporting procedures
- automation triggers for document numbering, workflow creation, audit logging, inspection-state propagation, and inventory posting
- sample data and demonstration queries

The SQL scripts were executed successfully on May 26, 2026 in a Docker-based Microsoft SQL Server 2022 environment. The live verification confirmed successful creation of the full schema, execution of the core procurement flow, automatic budget commitment creation, approval-step processing, quality inspection posting, and vendor claim creation.

7. User Interface

The user interface was implemented with Streamlit. The interface now contains ten functional modules.

7.1 Dashboard

Shows summary metrics for open requests, active suppliers, open orders, low-stock alerts, pending approvals, and open claims. It also displays current inventory and order receipt status.

7.2 Suppliers

Displays active suppliers, supplier contacts, the last quoted price function, and the required `RIGHT OUTER JOIN` query.

7.3 Purchase Requests

Displays open requests and allows the user to create a new purchase request by executing `usp_CreatePurchaseRequestWithItem`.

7.4 Offers and Orders

Displays supplier offer comparisons, exposes the ranking function, and allows conversion of an accepted offer into a purchase order by executing `usp_ApproveOfferAndCreateOrder`.

7.5 Warehouse Operations

Displays the required `LEFT OUTER JOIN`, reorder alerts, on-hand inventory lookup, and delivery receipt posting through `usp_RecordDeliveryAndReceipt`.

7.6 Administration

Displays audit logs, the SQL query gallery, and security objects for role-based review.

7.7 Approval Workflow

Displays pending approval steps and allows procurement or warehouse personnel to submit workflow decisions by executing `usp_SubmitApprovalDecision`.

7.8 Budget Tracking

Displays department-level budget usage, current commitments, remaining budget, and the budget transaction ledger.

7.9 Quality Control

Displays inspection summaries and allows the user to record inspection outcomes for a receipt item by executing `usp_RecordInspectionResult`.

7.10 Vendor Claims

Displays open supplier claims and allows creation of a vendor claim from an inspected receipt item by executing `usp_CreateVendorClaim`.

The user interface is therefore an execution layer for functions, views, and procedures rather than a static presentation shell.

8. Query Development

The project includes operational and reporting queries, with explicit support for the required outer join examples.

LEFT OUTER JOIN

Purpose: list all materials together with optional inventory records, even when no stock record exists for a warehouse-material combination.

Relations used:

- `Materials`
- `InventoryBalances`
- `Warehouses`

RIGHT OUTER JOIN

Purpose: list all suppliers, including suppliers that have not submitted an offer.

Relations used:

- `PurchaseOffers`
- `Suppliers`
- `BusinessPartners`

FULL OUTER JOIN

Purpose: compare request items and order items in a single result set and identify unmatched demand or unmatched order references.

Relations used:

- `PurchaseRequestItems`
- `PurchaseRequests`
- `PurchaseOrderItems`
- `PurchaseOrders`

In addition, the project includes reporting queries behind the new approval, budget, inspection, and claim modules through dedicated views.

9. Functions

The database includes seven functions.

1. `fn_GetInventoryOnHand` Returns current on-hand quantity for a material in a warehouse.
2. `fn_GetAvailableInventory` Returns on-hand minus reserved quantity.
3. `fn_GetRequestFulfillmentRate` Calculates request fulfillment percentage.
4. `fn_GetSupplierLastQuotedPrice` Returns the latest quoted price for a material from a supplier.
5. `fn_WarehouseReorderAlerts` Returns warehouse-material rows at or below reorder level.
6. `fn_RequestOfferRanking` Returns ranked quotation options for a request item.
7. `fn_GetRemainingBudget` Returns the remaining monthly budget for a department after applied budget transactions.

These functions are used directly by the UI and indirectly by reporting views.

10. Triggers

The system now includes fifteen triggers. Their purposes are:

1. `trg_PurchaseRequests_SetRequestNo` Generates request numbers.
2. `trg_PurchaseOffers_SetOfferNo` Generates offer numbers.
3. `trg_PurchaseOrders_SetOrderNo` Generates order numbers.
4. `trg_DeliveryReceipts_SetReceiptNo` Generates goods receipt numbers.
5. `trg_MaterialTransactions_SetTransactionNo` Generates material transaction numbers.
6. `trg_PurchaseRequests_CreateApprovalSteps` Creates approval instances for new requests.
7. `trg_PurchaseOrders_CreateApprovalSteps` Creates approval instances for new orders.
8. `trg_MaterialTransactionItems_ApplyInventory` Applies stock effects to inventory balances and blocks invalid stock states.
9. `trg_ApprovalSteps_SyncDocumentStatus` Synchronizes request or order document status after approval updates.
10. `trg_PurchaseOrders_AuditStatusChange` Writes order status transitions to the audit log.
11. `trg_Suppliers_SoftDelete` Converts supplier deletion into a soft-delete operation.
12. `trg_QualityInspections_SetInspectionNo` Generates inspection numbers.
13. `trg_QualityInspectionItems_UpdateReceiptStatus` Updates inspection status and related receipt status after inspection posting.
14. `trg_VendorClaims_SetClaimNo` Generates vendor claim numbers.

15. `trg_BudgetTransactions_AuditInsert` Writes budget transaction insert events to the audit log.

These triggers support automation, consistency, and operational traceability.

11. Views

The database includes nine major views.

1. `vw_ActiveSuppliers` Lists active suppliers and active-contact counts.
2. `vw_CurrentInventory` Shows current warehouse balances and available stock.
3. `vw_OpenPurchaseRequests` Lists open requests with fulfillment rate and item count.
4. `vw_SupplierOfferComparison` Shows supplier offer comparison and ranking.
5. `vw_OrderReceiptStatus` Shows order-line receipt status and accepted quantity.
6. `vw_PendingApprovals` Shows pending approval steps for requests and orders.
7. `vw_BudgetUsage` Shows budget amount, committed spend, remaining amount, and usage percentage by department and month.
8. `vw_InspectionSummary` Shows inspection outcomes by receipt item and material.
9. `vw_OpenVendorClaims` Shows open or submitted claims with supplier, inspection, and amount details.

These views reduce duplication in the UI and standardize reporting logic.

12. Transactions

The implementation contains six main stored procedures, with the first three representing the core document transactions and the latter three supporting control workflows.

12.1 `usp_CreatePurchaseRequestWithItem`

Creates a purchase request header and first request item atomically.

12.2 `usp_ApproveOfferAndCreateOrder`

Converts an offer into an order, creates order lines, updates request states, and creates a budget commitment when a budget exists.

12.3 `usp_RecordDeliveryAndReceipt`

Posts a goods receipt, updates order line receipt quantities, creates a stock transaction, and updates inventory through trigger-driven logic.

12.4 `usp_SubmitApprovalDecision`

Records a workflow decision for a request or order approval step and enforces sequential approval order.

12.5 `usp_RecordInspectionResult`

Creates an inspection header and inspection item for a receipt line while preventing over-inspection.

12.6 `usp_CreateVendorClaim`

Creates a vendor claim from an inspected receipt item and marks the inspection item as claimed.

Each procedure is wrapped in explicit transaction logic with rollback handling so that partial updates are avoided.

13. Concurrency Control

Concurrency handling was considered explicitly in the procedural layer.

13.1 Request Creation

`usp_CreatePurchaseRequestWithItem` uses `READ COMMITTED`. This prevents dirty reads during ordinary request entry while maintaining reasonable concurrency.

13.2 Offer Approval and Order Creation

`usp_ApproveOfferAndCreateOrder` uses `SERIALIZABLE` together with `UPDLOCK` and `HOLDLOCK`. This prevents duplicate conversion of the same offer into multiple orders and protects the budget-commitment step from race conditions.

13.3 Delivery Receipt Posting

`usp_RecordDeliveryAndReceipt` uses `REPEATABLE READ` and update locks on the relevant order item. This prevents lost updates when multiple users post receipts against the same order line.

13.4 Approval Decisions

`usp_SubmitApprovalDecision` uses `READ COMMITTED` with update locks on the target approval step. This prevents multiple decisions from being written simultaneously to the same approval row.

13.5 Inspection Posting

`usp_RecordInspectionResult` uses `REPEATABLE READ` and locks the receipt item during quantity validation. This prevents inspected quantity from exceeding received quantity under concurrent inspection activity.

13.6 Vendor Claim Creation

`usp_CreateVendorClaim` uses `READ COMMITTED` with update locks on the inspection item. This prevents multiple claim rows from consuming the same rejected or quarantine quantity inconsistently.

Together with triggers and check constraints, these isolation choices preserve integrity under simultaneous operations.

14. Inheritance

Inheritance is implemented with a supertype-subtype pattern:

- `BusinessPartners` is the supertype
- `Employees` is an employee subtype
- `Suppliers` is a supplier subtype

Common identity attributes are stored once in the supertype, while subtype-specific attributes remain in their own tables. This keeps the model extensible and avoids duplication of shared partner fields.

15. Privileges and Roles

Three database roles were created:

- `procurement_clerk`
- `warehouse_clerk`
- `reporting_analyst`

Three demo users without logins were also created:

- `procurement_demo`
- `warehouse_demo`
- `reporting_demo`

Privileges are assigned according to responsibility:

- procurement users can execute request, approval, order, budget, and claim procedures and read procurement-related views and functions
- warehouse users can execute receipt and inspection procedures and read inventory, inspection, and claim views
- reporting users receive read-only access to the analytical and audit layer

This design follows least-privilege principles while still making all project features demonstrable.

16. Additional Business Rules

Beyond the table structure itself, the implementation enforces the following business rules:

- inventory values cannot become negative

- reserved quantity cannot exceed on-hand quantity
- accepted quantity plus rejected quantity must equal received quantity at receipt posting
- inspection quantity cannot exceed the received quantity of the target receipt item
- claim quantity cannot exceed the rejected plus quarantined inspection quantity
- an offer cannot be converted into an order more than once
- approval steps must be completed in sequence
- a department budget commitment cannot be inserted if the remaining budget is insufficient
- supplier deletion is soft delete rather than physical deletion
- request, offer, order, inspection, and claim numbering is generated automatically
- document and workflow statuses are restricted by check constraints

These rules ensure that the database behaves like an operational business system rather than a passive collection of tables.

17. SQL Statement Inventory

The project includes the following SQL statement categories:

- `IF DB_ID(...) IS NOT NULL`
- `DROP DATABASE`
- `CREATE DATABASE`
- `USE`
- `CREATE TABLE`
- `CREATE INDEX`
- `CREATE UNIQUE INDEX`
- `CREATE ROLE`
- `CREATE USER`
- `ALTER ROLE ... ADD MEMBER`
- `GRANT`
- `INSERT`
- `UPDATE`
- `DELETE` through trigger-controlled soft-delete logic
- `CREATE OR ALTER FUNCTION`
- `CREATE OR ALTER VIEW`
- `CREATE OR ALTER TRIGGER`
- `CREATE OR ALTER PROCEDURE`
- `BEGIN TRAN`, `COMMIT`, `ROLLBACK`
- `SELECT` statements, including `LEFT OUTER JOIN`, `RIGHT OUTER JOIN`, and `FULL OUTER JOIN`

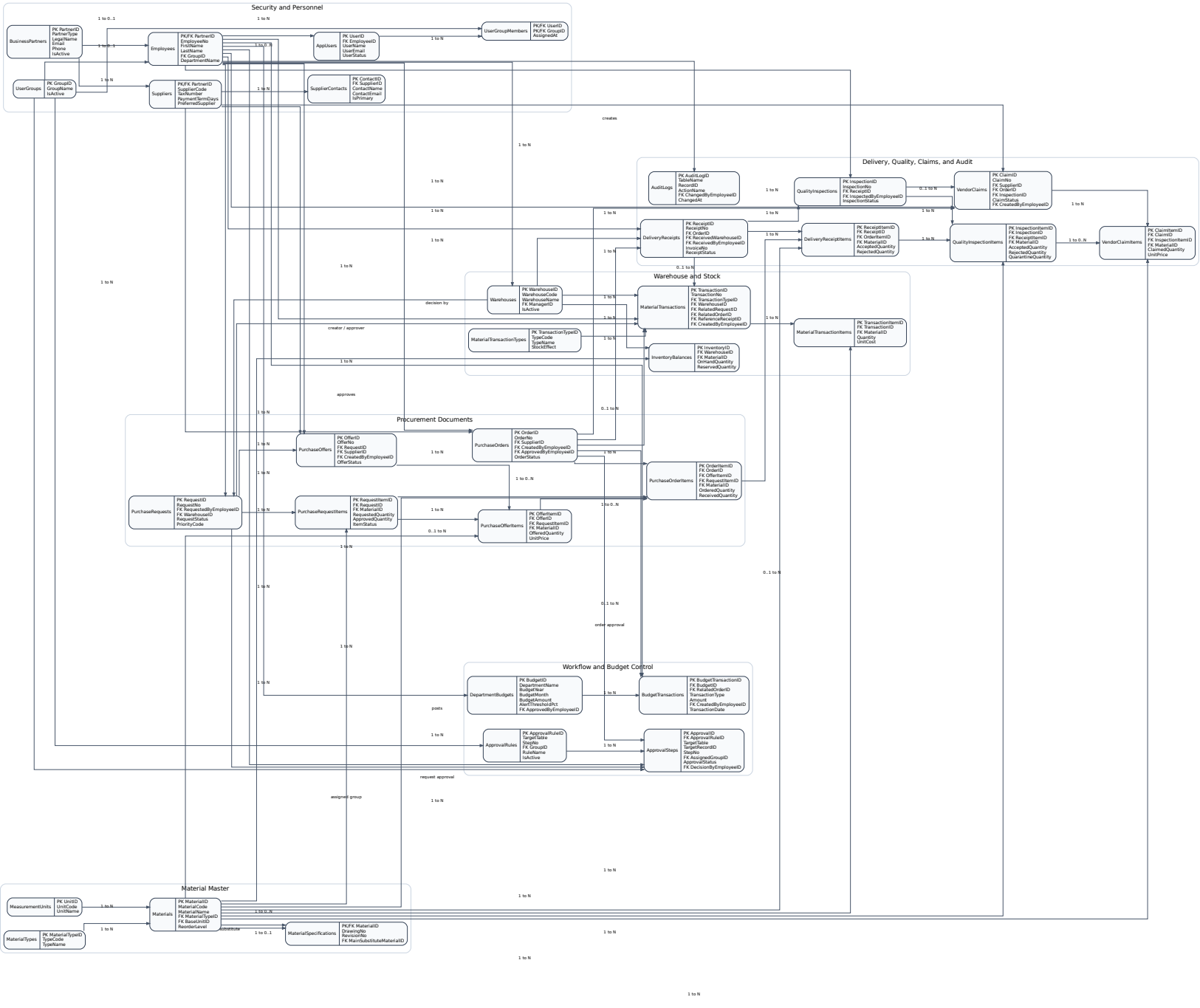
Conclusion

The final system combines a normalized relational schema, a layered procurement-to-warehouse workflow, approval and budget control, quality inspection processing, vendor claim tracking, role-based security, and a multi-module Streamlit interface. The implementation is not limited to

schema definition: it also includes live transactional behavior, automation triggers, reporting views, reusable functions, and successful execution on a real SQL Server 2022 environment.

Appendix A. Consolidated Full E-R Diagram

The complete integrated E-R diagram is presented below as a single figure. Because the full schema is dense, enlarged overlapping panels follow in the next appendix.


Security and Personnel
BusinessPartners
PK PartnerID
PartnerType
LegalName
Email
Phone
IsActive
Employees
PK/FK PartnerID
EmployeeNo
FirstName
LastName
FK GroupID
DepartmentName
AppUsers
PK UserID
FK EmployeeID
UserName
UserEmail
UserStatus
UserGroupMembers
PK/FK UserID
PK/FK GroupID
AssignedAt
UserGroups
PK GroupID
GroupName
IsActive
Suppliers
PK/FK PartnerID
SupplierCode
TaxIDNumber
PaymentTermDays
PreferredSupplier
SupplierContacts
PK ContactID
FK SupplierID
ContactName
ContactEmail
IsPrimary
1 to 0..1
1 to N
creates
Delivery, Quality, Claims, and Audit
AuditLogs
PK AuditLogID
TableName
RecordID
ActionName
FK ChangedByEmployeeID
ChangedAt
QualityInspections
PK InspectionID
InspectionNo
FK ReceiptID
FK InspectedByEmployeeID
InspectionStatus
VendorClaims
PK ClaimID
ClaimNo
FK SupplierID
FK OrderID
FK InspectionID
ClaimStatus
FK CreatedByEmployeeID
DeliveryReceipts
PK ReceiptID
ReceiptNo
FK OrderID
FK ReceivedWarehouseID
FK ReceivedByEmployeeID
InvoiceNo
ReceiptStatus
DeliveryReceiptItems
PK ReceiptItemID
FK ReceiptID
FK OrderItemID
FK MaterialID
AcceptedQuantity
RejectedQuantity
QualityInspectionItems
PK InspectionItemID
FK InspectionID
FK ReceiptItemID
FK MaterialID
AcceptedQuantity
RejectedQuantity
QuarantineQuantity
VendorClaimItems
PK ClaimItemID
FK ClaimID
FK InspectionItemID
FK MaterialID
ClaimedQuantity
UnitPrice
0..1 to N
Warehouse and Stock
Warehouses
PK WarehouseID
WarehouseCode
WarehouseName
FK ManagerID
IsActive
MaterialTransactions
PK TransactionID
TransactionNo
FK TransactionTypeID
FK WarehouseID
FK RelatedRequestID
FK RelatedOrderID
FK ReferenceReceiptID
FK CreatedByEmployeeID
MaterialTransactionItems
PK TransactionItemID
FK TransactionID
FK MaterialID
Quantity
UnitCost
MaterialTransactionTypes
PK TransactionTypeID
TypeCode
TypeName
StockEffect
InventoryBalances
PK InventoryID
FK WarehouseID
FK MaterialID
OnHandQuantity
ReservedQuantity
decision by
creator / approver
approves
Procurement Documents
PurchaseOffers
PK OfferID
OfferNo
FK RequestID
FK SupplierID
FK CreatedByEmployeeID
OfferStatus
PurchaseOrders
PK OrderID
OrderNo
FK SupplierID
FK CreatedByEmployeeID
FK ApprovedByEmployeeID
OrderStatus
PurchaseOrderItems
PK OrderItemID
FK OrderID
FK OfferItemID
FK RequestItemID
FK MaterialID
OrderedQuantity
ReceivedQuantity
PurchaseRequests
PK RequestID
RequestNo
FK RequestedByEmployeeID
FK WarehouseID
RequestStatus
PriorityCode
PurchaseRequestItems
PK RequestItemID
FK RequestID
FK MaterialID
RequestedQuantity
ApprovedQuantity
ItemStatus
PurchaseOfferItems
PK OfferItemID
FK OfferID
FK RequestItemID
FK MaterialID
OfferedQuantity
UnitPrice
0..1 to N
order approval
Workflow and Budget Control
DepartmentBudgets
PK BudgetID
DepartmentName
BudgetYear
BudgetMonth
BudgetAmount
AlertThresholdPct
FK ApprovedByEmployeeID
BudgetTransactions
PK BudgetTransactionID
FK BudgetID
FK RelatedOrderID
TransactionType
Amount
FK CreatedByEmployeeID
TransactionDate
ApprovalRules
PK ApprovalRuleID
TargetTable
StepNo
FK GroupID
RuleName
IsActive
ApprovalSteps
PK ApprovalID
FK ApprovalRuleID
TargetTable
TargetRecordID
StepNo
FK AssignedGroupID
ApprovalStatus
FK DecisionByEmployeeID
posts
request approval
assigned group
Material Master
MeasurementUnits
PK UnitID
UnitCode
UnitName
MaterialTypes
PK MaterialTypeID
TypeCode
TypeName
Materials
PK MaterialID
MaterialCode
MaterialName
FK MaterialTypeID
FK BaseUnitID
ReorderLevel
MaterialSpecifications
PK/FK MaterialID
DrawingNo
RevisionNo
FK MainSubstituteMaterialID
substitute
1 to 0..N
1 to 0..1

Appendix B. Enlarged Full-Diagram Panels

The following four pages show enlarged overlapping panels of the same integrated diagram to improve readability of entity names and relationship labels.

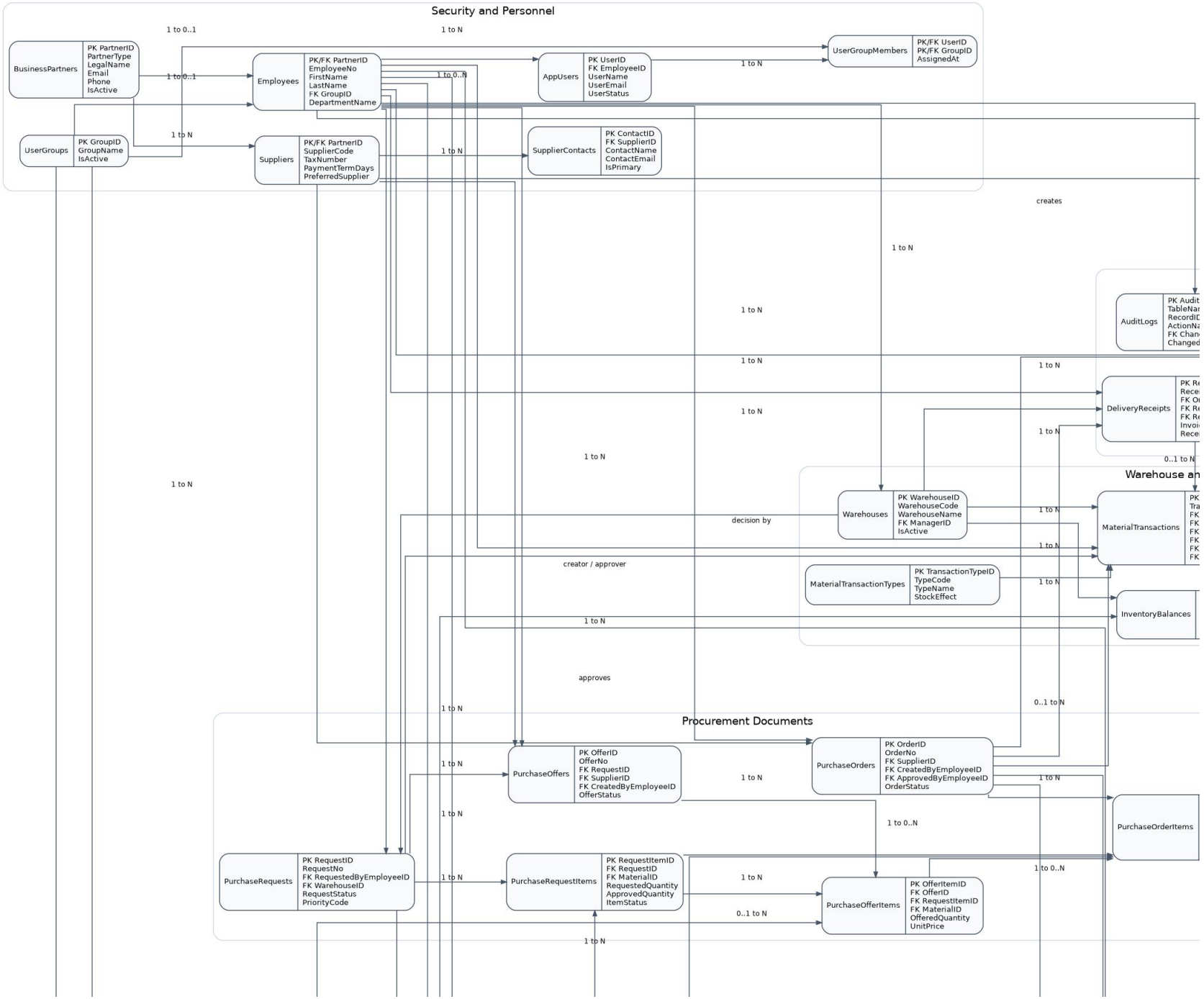


Figure 6: Enhanced model 1 of the integrated E-Policy

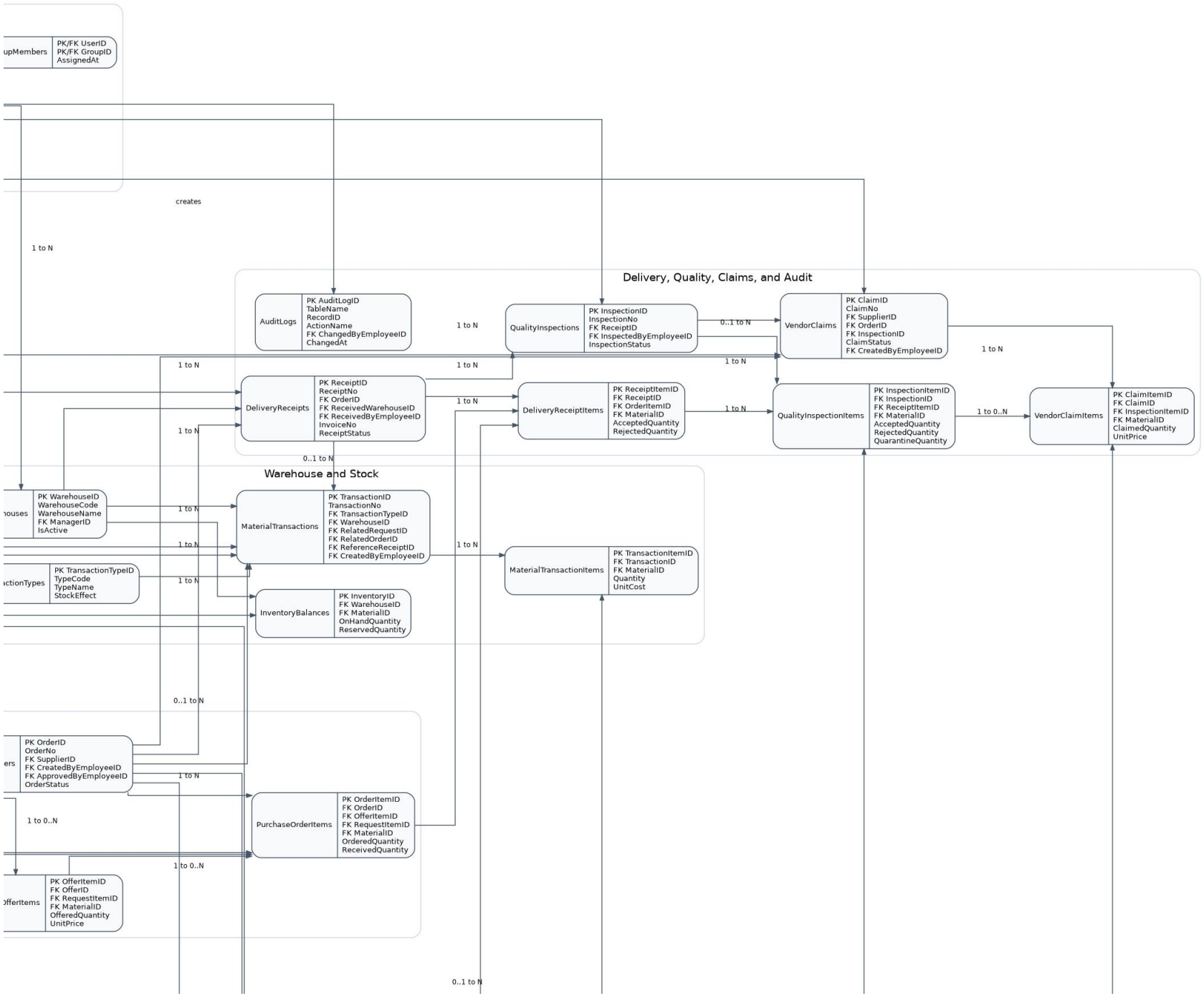
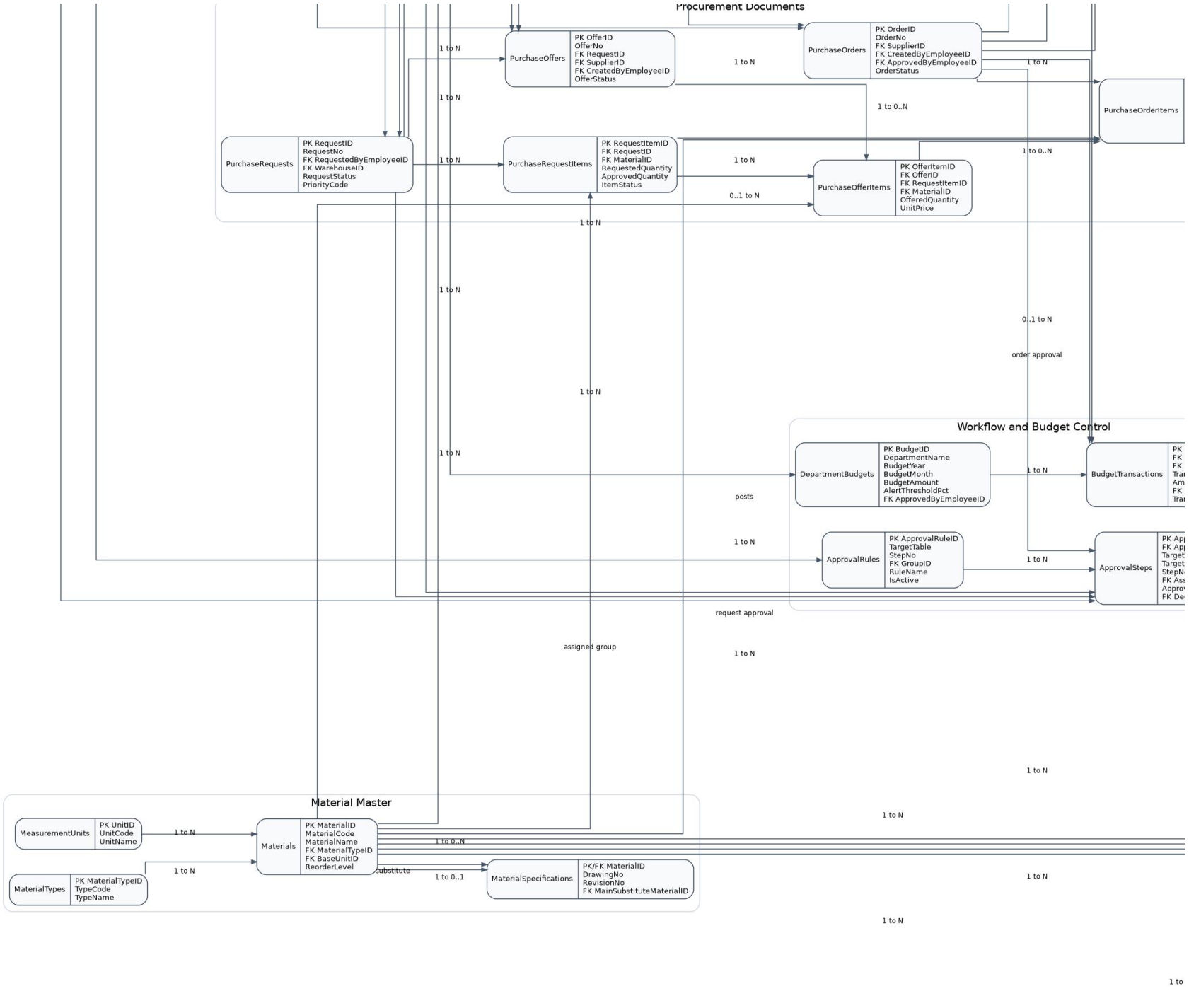


Figure 7. Enhanced model 2 of the integrated E-R diagram


Procurement Documents
PurchaseOffers
PK OfferID
OfferNo
FK RequestID
FK SupplierID
FK CreatedByEmployeeID
OfferStatus
PurchaseOrders
PK OrderID
OrderNo
FK SupplierID
FK CreatedByEmployeeID
FK ApprovedByEmployeeID
OrderStatus
PurchaseOrderItems
PurchaseRequests
PK RequestID
RequestNo
FK RequestedByEmployeeID
FK WarehouseID
RequestStatus
PriorityCode
PurchaseRequestItems
PK RequestItemID
FK RequestID
FK MaterialID
RequestedQuantity
ApprovedQuantity
ItemStatus
PurchaseOfferItems
PK OfferItemID
FK OfferID
FK RequestItemID
FK MaterialID
OfferedQuantity
UnitPrice
1 to N
1 to 0..N
0..1 to N
order approval
Workflow and Budget Control
DepartmentBudgets
PK BudgetID
DepartmentName
BudgetYear
BudgetMonth
BudgetAmount
AlertThresholdPct
FK ApprovedByEmployeeID
BudgetTransactions
posts
ApprovalRules
PK ApprovalRuleID
TargetTable
StepNo
FK GroupID
RuleName
IsActive
ApprovalSteps
request approval
assigned group
Material Master
MeasurementUnits
PK UnitID
UnitCode
UnitName
Materials
PK MaterialID
MaterialCode
MaterialName
FK MaterialTypeID
FK BaseUnitID
ReorderLevel
MaterialTypes
PK MaterialTypeID
TypeCode
TypeName
substitute
MaterialSpecifications
PK/FK MaterialID
DrawingNo
RevisionNo
FK MainSubstituteMaterialID

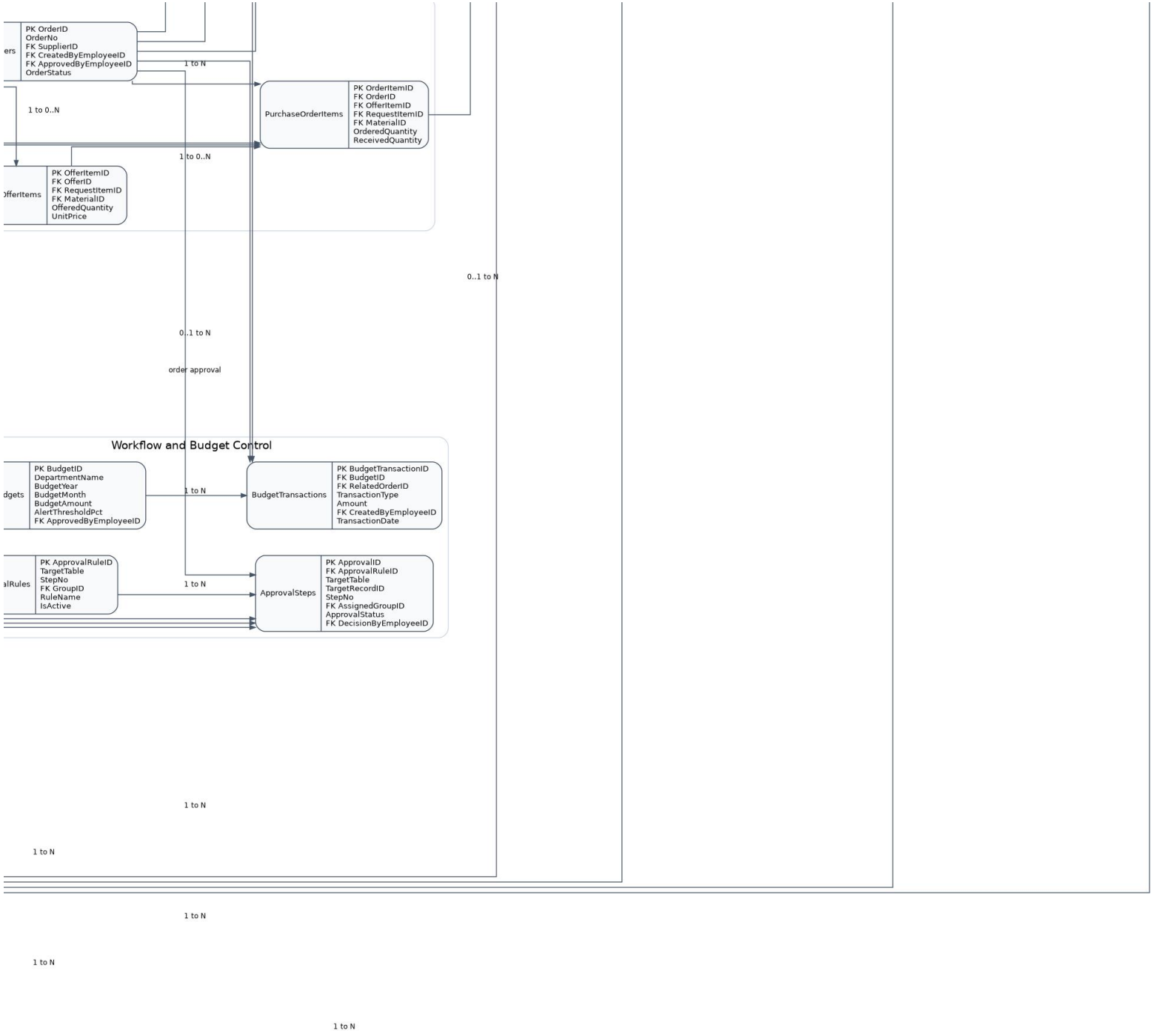


Figure 9. Enhanced model 4 of the integrated E-Policy